CS 301: High-Performance Computing

Assignment 04 - Particle Interpolation and Mover Performance Analysis

Darshit Ramani(202301404)

Vasu Bhensdadia(202301416)

March 11, 2026

Contents

1 Problem Description **3**

2 Graphs **3**

3 Flow diagram **9**

4 Speedup Analysis of OpenMP Parallelization **9**

5 Parallelization of the Interpolation Function **10**

6 Conclusion **11**

1 Problem Description

This lab investigates the performance of a scattered particle-to-mesh interpolation algorithm and a stochastic particle mover operation within a 1×1 computational domain. The objective is to analyze how interpolation execution time scales with increasing numbers of particles and different grid configurations. Additionally, the experiment evaluates the consistency of interpolation performance across multiple iterations. In the final stage, a particle mover operation is introduced and parallelized using OpenMP to study the performance improvement achieved through shared-memory parallelism. The results obtained from the Lab PC and the HPC cluster are compared to analyze computational efficiency and scalability.

2 Graphs

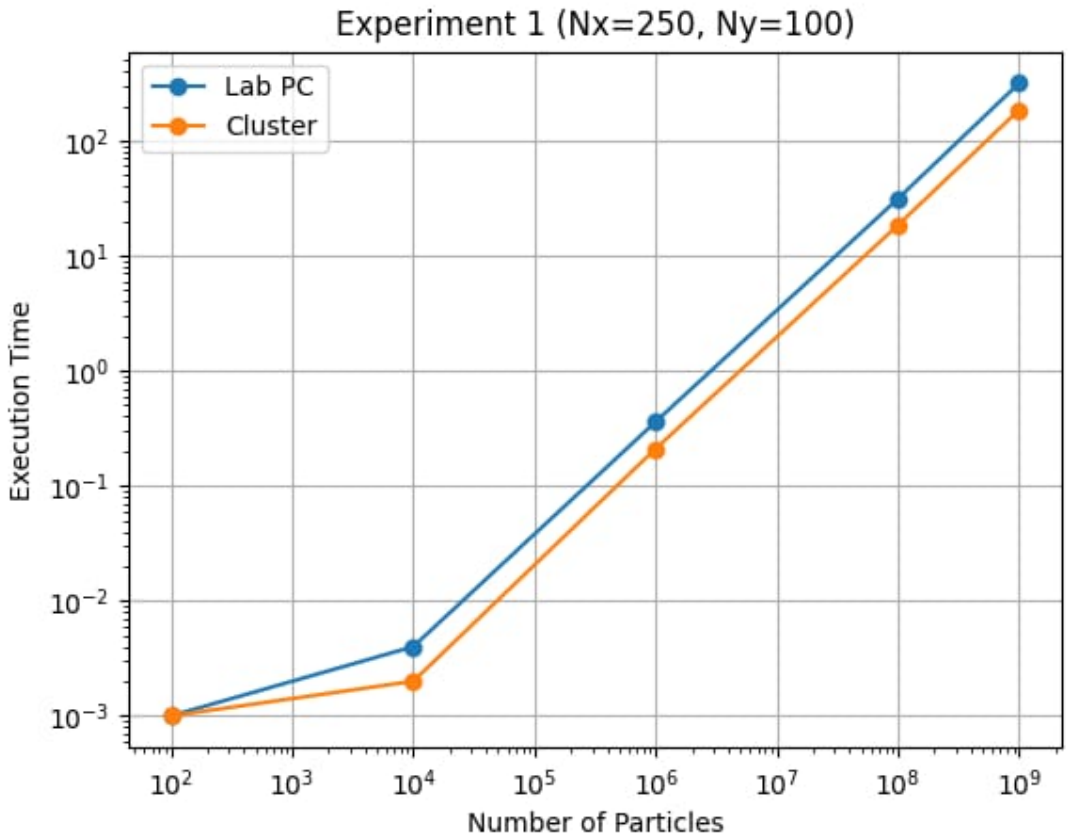

Figure 1: Nx = 250, Ny = 100

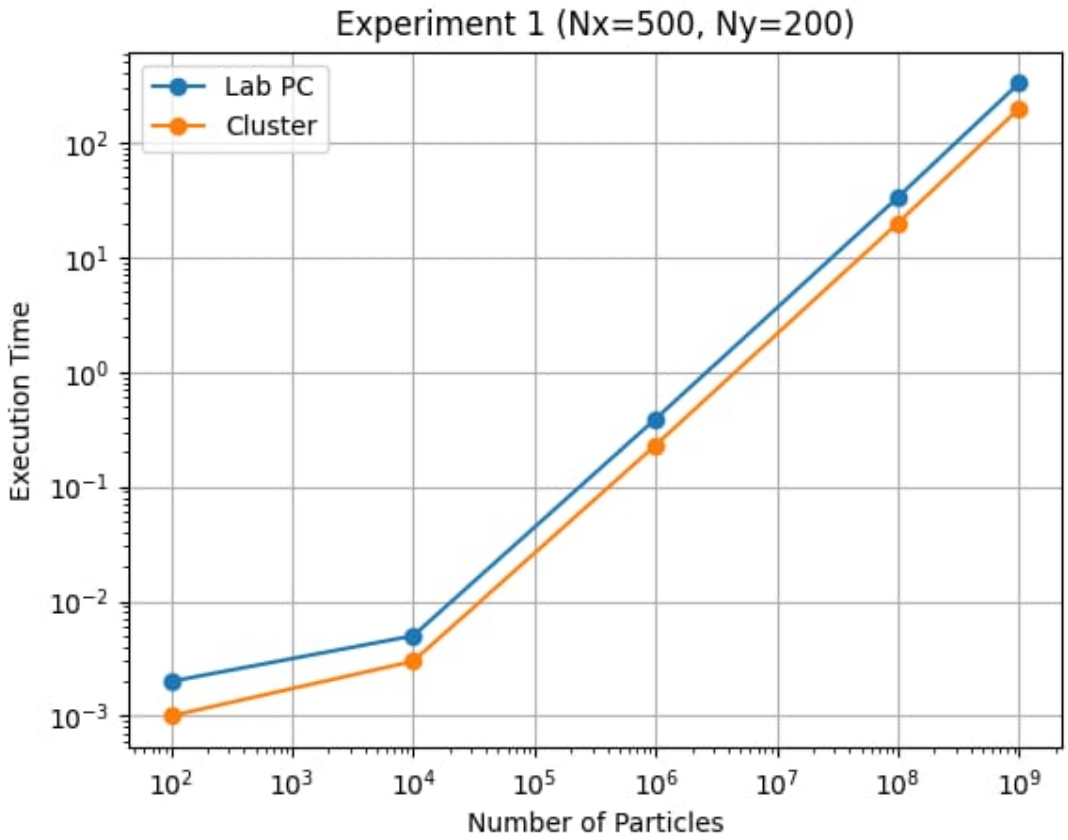

Figure 2: Nx = 500, Ny = 200

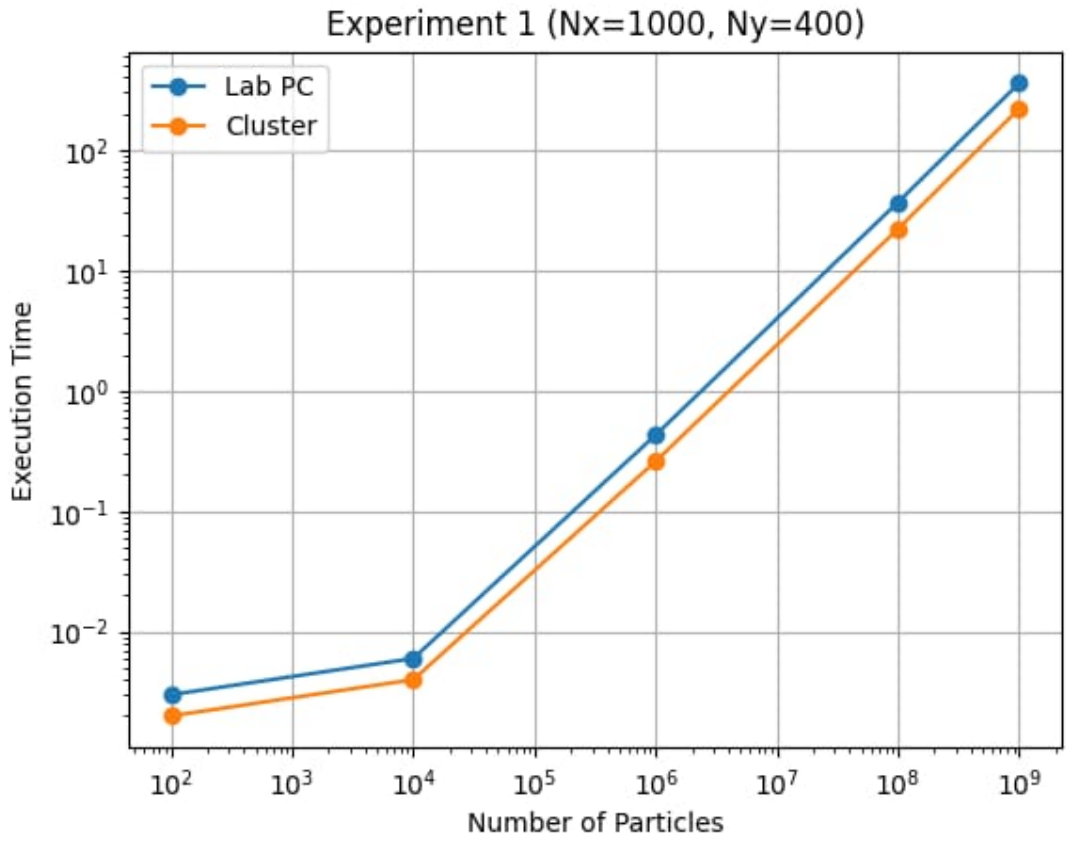

Figure 3: Nx = 1000, Ny = 400

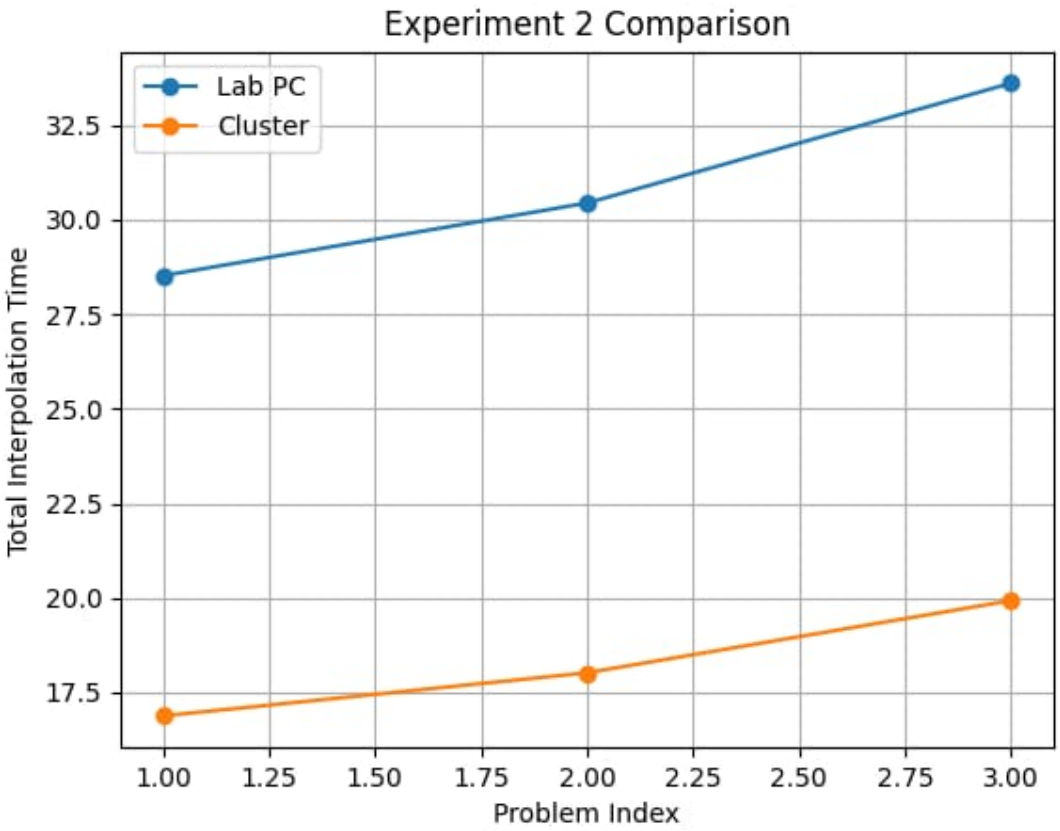

Figure 4: index vs interpolation time

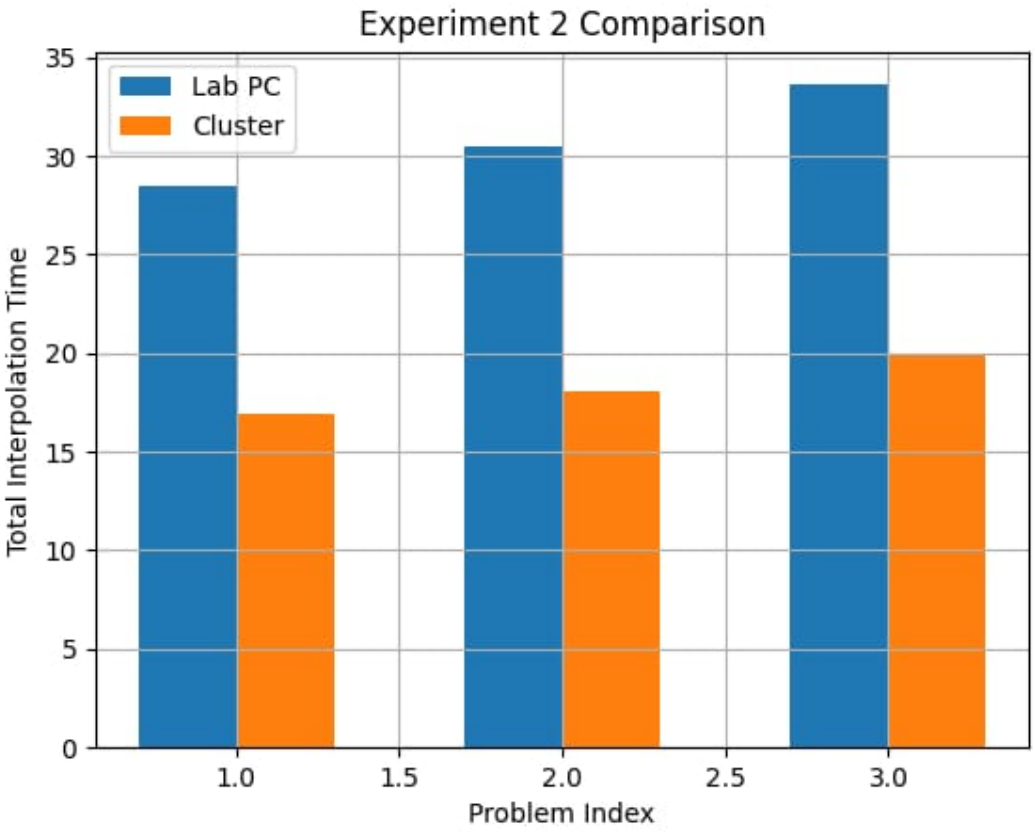

Figure 5: Bar plo

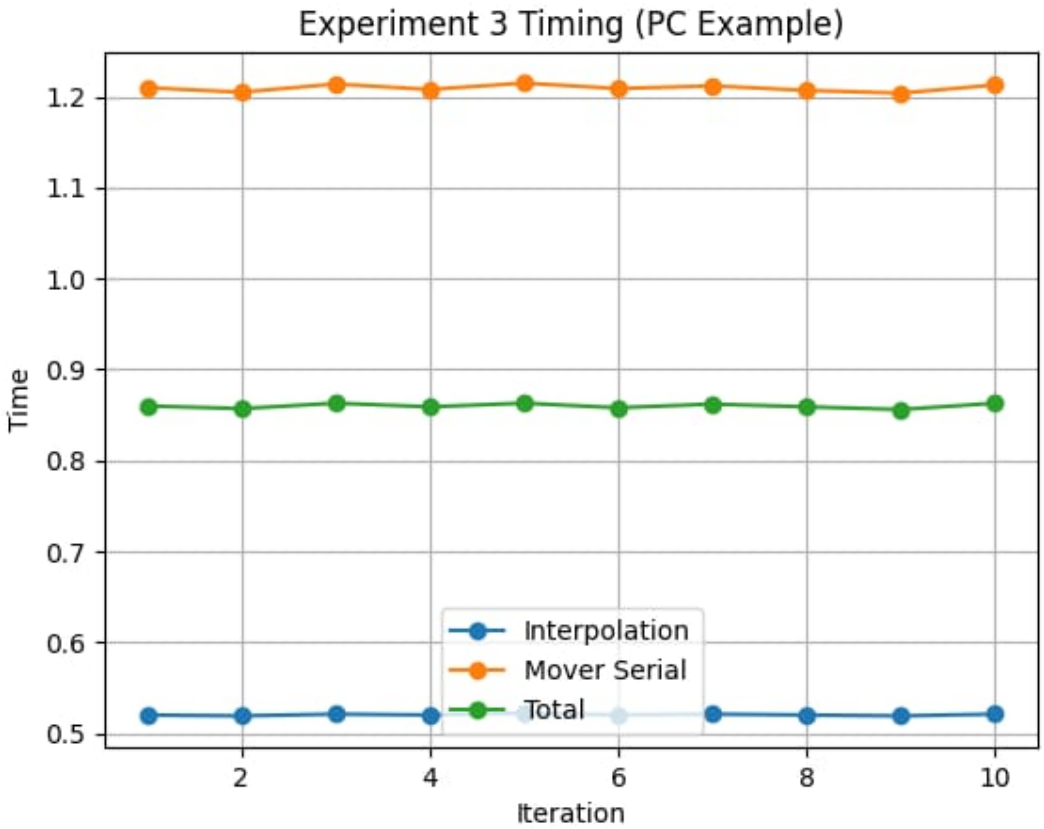

Figure 6: Iteration timing plot

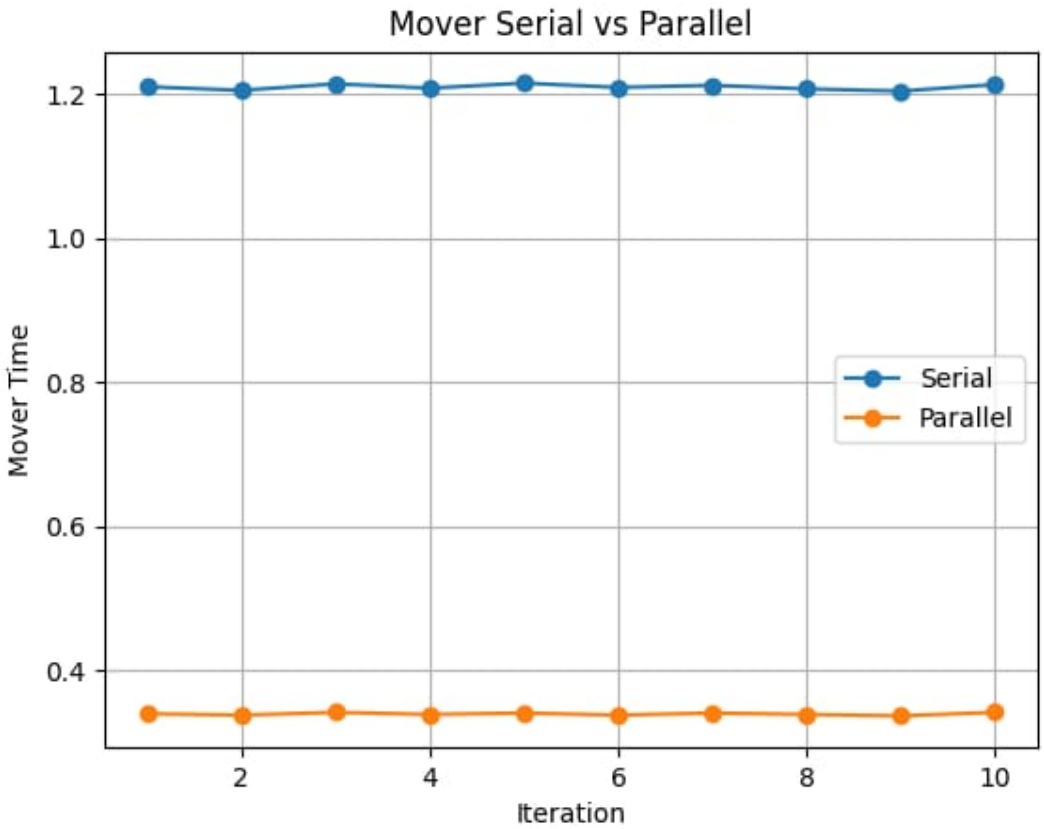

Figure 7: Serial vs Parallel

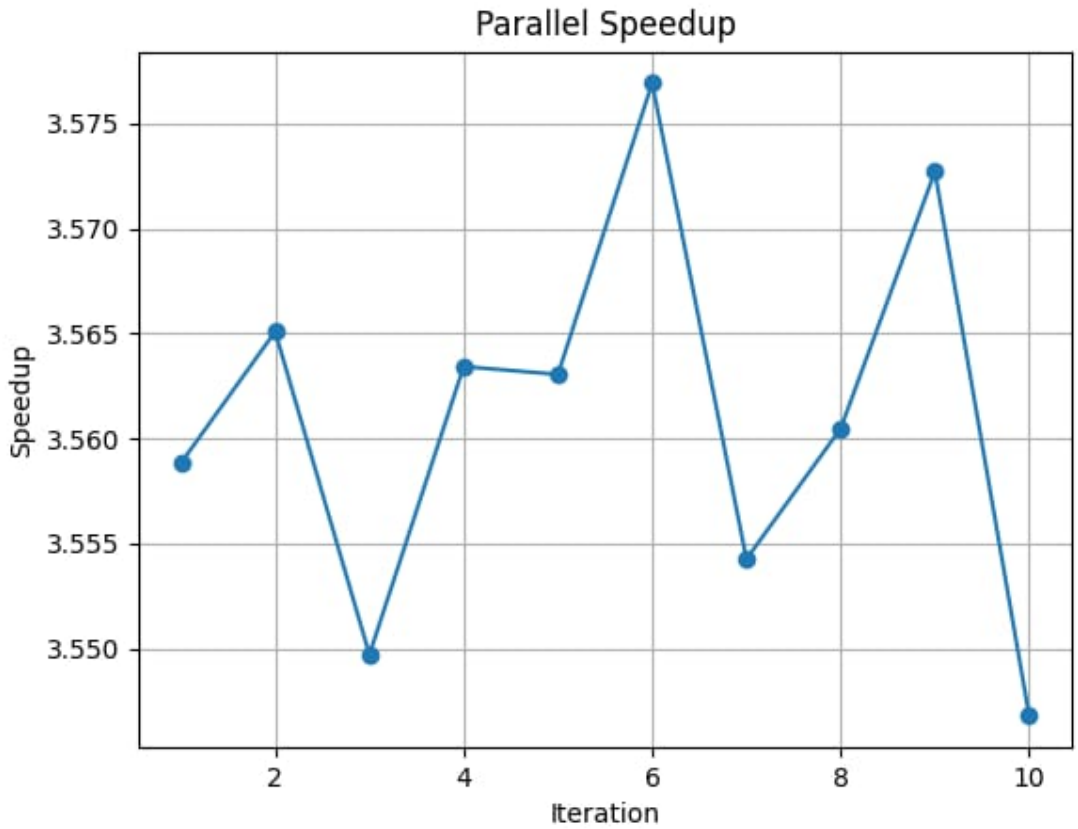

Figure 8: Speedup

3 Flow diagram

4 Speedup Analysis of OpenMP Parallelization

The speedup achieved by the OpenMP parallelization of the mover operation is calculated using the following relation:

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

where T_{serial} represents the execution time of the serial mover implementation and T_{parallel} represents the execution time of the parallel mover implementation using OpenMP.

From the experimental results, the average serial mover execution time on the Lab PC is approximately 1.21 seconds, while the parallel implementation using 4 threads takes approximately 0.34 seconds. Therefore, the achieved speedup is:

$$S = \frac{1.21}{0.34} \approx 3.56$$

This speedup is close to the theoretical maximum of 4 for a system using four threads. However, perfect linear speedup is not achieved due to several hardware-related factors such as thread management overhead, memory bandwidth limitations, and cache contention between threads. Additionally, synchronization and scheduling overhead introduced by OpenMP also contribute to the deviation from ideal performance.

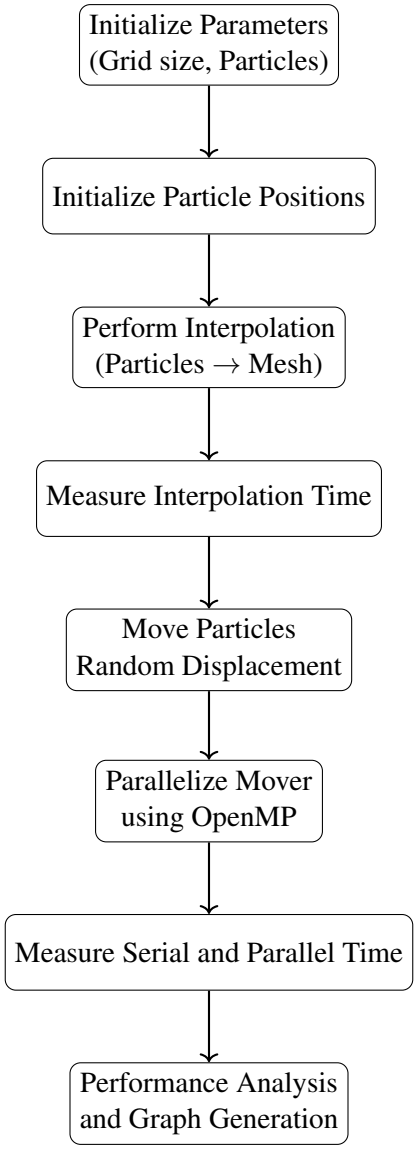

Figure 9: Overall workflow of the particle interpolation and mover performance analysis.

Overall, the results demonstrate that the mover operation benefits significantly from parallelization since particle updates are independent of each other, making the algorithm well suited for shared-memory parallel execution.

5 Parallelization of the Interpolation Function

If the interpolation function were to be parallelized, a straightforward approach would be to distribute the particle processing loop across multiple threads using OpenMP. Since each particle contributes to the value of a grid cell based on its spatial location, the particle list can be divided among threads so that each thread processes a subset of particles concurrently.

However, a key challenge arises due to potential data races. When multiple particles fall into the same or neighboring grid cells, multiple threads may attempt to update the same mesh location simultaneously. This leads to concurrent writes to shared memory, which can result in incorrect values if proper synchronization is not used.

To avoid this issue, synchronization mechanisms such as atomic operations or critical sections could be used when updating mesh values. Alternatively, a more efficient approach would be to assign each

thread a private copy of the mesh grid and combine the results in a final reduction step. While these approaches ensure correctness, they may introduce additional overhead or increase memory usage.

Therefore, the main bottleneck in parallelizing the interpolation function is the contention for shared grid updates. This can limit scalability and reduce the efficiency of parallel execution compared to the mover operation, where particle updates are independent and do not require shared memory writes.

6 Conclusion

In this study, the performance of a particle-to-mesh interpolation algorithm and a stochastic particle mover operation was analyzed under different grid configurations and particle counts. The results show that the interpolation time increases approximately linearly with the number of particles, indicating good computational scalability. Furthermore, the comparison between the Lab PC and the HPC cluster demonstrates the impact of improved hardware resources on overall execution time.

The mover operation was successfully parallelized using OpenMP, achieving a speedup close to the theoretical limit for four threads. This highlights that operations with independent particle updates are well suited for shared-memory parallelism. However, parallelizing the interpolation function is more challenging due to potential race conditions when multiple particles attempt to update the same grid cell. Overall, the experiments demonstrate the importance of algorithm structure and memory access patterns in achieving efficient parallel performance.